

# AccelMPC: High-Rate, Low-Power FPGA-Accelerated Model Predictive Control for Tiny Drones

Anonymous Author A<sup>1</sup>, Anonymous Author B<sup>2</sup>

**Abstract**—Unlocking the potential of tiny aerial robots requires order of magnitude improvements in the performance of embedded edge control. In particular, although recent cached model predictive control (MPC) solvers can handle the fast system dynamics and complex constraints required for agile drone flight, their computational demands remain prohibitive for resource-constrained robots, forcing prior implementations to operate at reduced control rates. AccelMPC overcomes this challenge through an end-to-end co-design approach that jointly optimizes the solver algorithm, numerical representation, hardware mapping, and physical integration. AccelMPC pairs a co-designed FPGA-accelerated alternating direction method of multipliers (ADMM)-based MPC solver with a custom 6 g PCB, providing high-bandwidth communication for deployment on a 35 g Crazyflie. Hardware experiments demonstrate 1 kHz onboard constrained MPC with dynamic obstacles, up to  $15.6\times$  faster solve times and  $195.4\times$  improvement in energy-delay product over state-of-the-art embedded microcontroller-based solvers, all while scaling to optimization problems with over 20,000 optimization variables and a comparable number of constraints. We release our PCB design files, firmware, and FPGA solver code open source.

## I. INTRODUCTION

Tiny aerial robots are a promising solution for applications ranging from emergency search and rescue to routine monitoring and maintenance of infrastructure and equipment [1], [2], [3]. However, despite advances in large-scale autonomous drone control [4], [5], the onboard computational budgets in terms of size, weight, and power (SWaP) severely constrain the performance of such tiny systems [6]. This is a particular challenge given that the fast and unstable dynamics of such systems demand feedback rates approaching the kilohertz regime for complex tasks.

Model Predictive Control (MPC) [7], [8], [9] has been used in prior aerial robotics work to deliver such high-performance feedback control while explicitly accounting for system dynamics and constraints, enabling constrained trajectory tracking, collision avoidance, perception-aware control, learned high-speed models, and time-optimal flight [10], [11], [12], [13], [14], [15]. Most importantly, recent advances in cached MPC, such as TinyMPC [16], demonstrate how real-time operation can be achieved on the limited compute platforms available onboard tiny drones. However, at kilohertz control rates, the alternating direction method of multipliers (ADMM)-based [17] solvers in those works can complete only one or two iterations per control step, which is insufficient for reliable convergence when constraints are active, thus requiring operation at reduced control

rates [16], [18], [19], [20]. Consequently, a fundamental gap remains between the predictive, explicitly constrained control promised by MPC and the capabilities available onboard highly resource-constrained robots.

AccelMPC closes this gap through end-to-end hardware-algorithm co-design, jointly optimizing solver structure, numerical representation, hardware mapping, and physical integration onboard our target tiny-drone platform. AccelMPC pairs a co-designed FPGA-accelerated ADMM-based MPC solver with a custom 6 g PCB that provides high-bandwidth communication for real-time robot deployments. We realize the same solver through two complementary FPGA mappings, exposing the hardware trade-off between energy-efficient high-rate control and scalability to substantially larger optimization problems.

Overall, this tight co-design enables 1 kHz fully onboard constrained MPC with runtime-updated bounds, demonstrated through dynamic obstacle avoidance on a 35 g Crazyflie quadrotor (Fig. 1). This represents up to a  $15.6\times$  faster solve time and  $195.4\times$  improvement in energy-delay product over state-of-the-art embedded solvers, while supporting more than 20,000 optimization variables and a comparable number of constraints.

Together, these results establish FPGA hardware-algorithm co-design as an enabling approach for high-bandwidth, explicitly constrained predictive control on highly compute-limited robotic platforms. We release our project open-source, including PCB design files, firmware, and FPGA solver code at: redacted for review.

## II. BACKGROUND

### A. Linear MPC Formulation

We consider the following linear MPC problem with a prediction horizon  $H$ , states  $x_k \in \mathbb{R}^n$ , and controls  $u_k \in \mathbb{R}^m$ , where  $A$  and  $B$  define the linear system dynamics,  $Q \succeq 0$ ,  $Q_f \succeq 0$ , and  $R \succ 0$  define the quadratic cost, and the constraint sets  $\mathcal{X}$  and  $\mathcal{U}$  are assumed polyhedral:

$$\begin{aligned} \min_{\{x_k, u_k\}} \quad & \sum_{k=0}^{H-1} [(x_k - x_{\text{ref}})^\top Q (x_k - x_{\text{ref}}) \\ & + (u_k - u_{\text{ref}})^\top R (u_k - u_{\text{ref}})] \\ & + (x_H - x_{\text{ref}})^\top Q_f (x_H - x_{\text{ref}}) \\ \text{s.t.} \quad & x_{k+1} = Ax_k + Bu_k, \quad k = 0, \dots, H-1, \\ & x_0 = \hat{x}, \quad x_k \in \mathcal{X}, \quad u_k \in \mathcal{U}. \end{aligned} \quad (1)$$

By stacking states and inputs into an interleaved vector,

$$w = [x_0^\top, u_0^\top, x_1^\top, u_1^\top, \dots, x_{H-1}^\top, u_{H-1}^\top, x_H^\top]^\top, \quad (2)$$

<sup>1</sup> Anonymous Institution A

<sup>2</sup> Anonymous Institution B

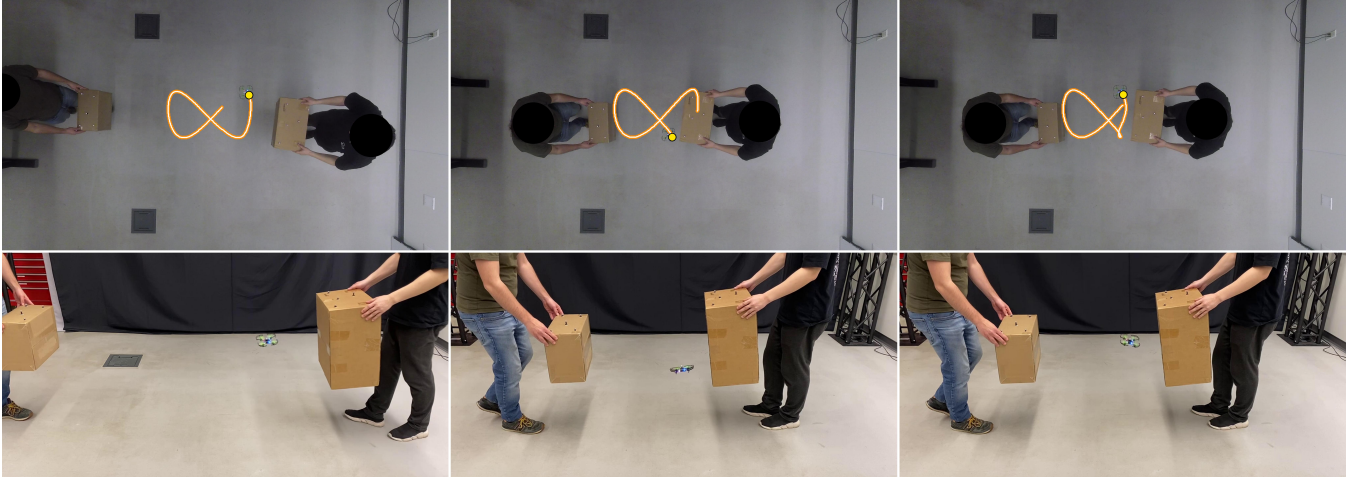


Fig. 1: 1 kHz onboard dynamic obstacle avoidance. From left to right: (a) nominal trajectory before interaction, (b) the quadrotor approaches a moving obstacle, and (c) the quadrotor adapts its trajectory online to safely avoid the obstacle.

we can write (1) as a sparse convex quadratic program:

$$\min_w \frac{1}{2} w^\top P w + c^\top w \quad \text{s.t.} \quad l \leq M w \leq u, \quad (3)$$

where  $P \succeq 0$  is block-diagonal,  $M$  is sparse and block-banded due to the stage-wise dynamics coupling, and reference tracking is encoded by the linear term  $c$ . Equality constraints are encoded as rows of (3) with  $l_i = u_i$ . The interleaved ordering of  $w$  keeps all coupling local to each stage, generating a banded matrix structure we exploit in Sec. IV-A. We assume that the matrices  $P$  and  $M$  are fixed, while the vectors  $l$  and  $u$  can be updated at runtime.

### B. ADMM for Solving the Linear MPC QP

We review ADMM [17], as applied to (3). Introducing the auxiliary variable  $y \in \mathbb{R}^m$  results in the following equivalent problem where  $w \in \mathbb{R}^n$ ,  $M \in \mathbb{R}^{m \times n}$ , and  $I_C(y)$  is the indicator function of the convex set  $\mathcal{C} = \{y \mid l \leq y \leq u\}$ :

$$\min_{w,y} \frac{1}{2} w^\top P w + c^\top w + I_C(y) \quad \text{s.t.} \quad M w = y. \quad (4)$$

The augmented Lagrangian of this problem is as follows where the dual variable  $\lambda \in \mathbb{R}^m$  is the unscaled Lagrange multiplier for the constraint  $M w - y = 0$ :

$$\mathcal{L}_\rho(w, y, \lambda) = \frac{1}{2} w^\top P w + c^\top w + I_C(y) + \frac{\rho}{2} \|M w - y\|_2^2 + \lambda^\top (M w - y). \quad (5)$$

We can then solve this via three-step ADMM iterations:

$$w^+ = \arg \min_w \mathcal{L}_\rho(w, y, \lambda), \quad (6a)$$

$$y^+ = \Pi_{[l,u]} \left( M w^+ + \frac{1}{\rho} \lambda \right), \quad (6b)$$

$$\lambda^+ = \lambda + \rho (M w^+ - y^+), \quad (6c)$$

where  $^+$  denotes the next iterate,  $\Pi_{[l,u]}(\cdot)$  is the projection onto the box  $[l, u]$  and the  $w$ -update reduces to the structured linear system solve:

$$(P + \rho M^\top M) w^+ = -c + \rho M^\top y - M^\top \lambda, \quad (7)$$

where  $M^\top \lambda$  maps the constraint-space dual variable into the primal space of the linear-system right-hand side. As discussed in Section IV-A, this structured solve is particularly amenable to hardware acceleration. In our implementation, we use the scaled dual variable  $s = \lambda/\rho$  and maintain  $b = \rho M^\top (y - s)$ , so that the linear-system right-hand side is  $b - c$ .

## III. RELATED WORK

### A. Embedded and Edge MPC Solvers

Given the structured QP formulation described in Sec. II, a wide range of embedded and edge MPC solvers have been proposed with different trade-offs [21]. Popular approaches include code-generation tools such as CVXGEN [22] which produce solvers tailored to a specific QP structure, enabling efficient execution on edge CPUs. More general-purpose approaches, such as OSQP [23], rely on operator-splitting methods with warm-starting and have been widely adopted in robotics. Structured solvers and frameworks such as HPIPM [24] and acados [25] further exploit problem structure and optimized linear algebra to achieve high performance on embedded platforms, while robotics-oriented solvers such as ALTRO-C target fast conic MPC on more capable onboard computers [26]. Another thread has investigated GPU-accelerated MPC, exploiting data-level parallelism for high-performance [27], [28], [29].

Tiny quadrotors like the Crazyflie, however, can support neither power-hungry edge CPUs nor GPUs, and instead rely on resource-constrained microcontrollers (MCUs). Recent advances in cached MPC, such as TinyMPC [16], provide the first glimpses into the power of real-time MPC on-board tiny platforms like the Crazyflie [30]. These approaches leverage ADMM to reformulate the primal update as an unconstrained LQR problem and cache the associated Riccati factorizations offline, enabling efficient QP solves within tight memory and timing constraints. However, when constraints become active and the number of ADMM iterations required for convergence grows, even these approaches are unable to scale on MCUs, requiring operation at reduced control rates.

### B. FPGA-Based Linear MPC

A significant body of work has explored FPGA acceleration of linear MPC [31]: Jerez *et al.* [32], [33] demonstrated fixed-point first-order methods and sparse QP solvers at very high rates; Hartley *et al.* [34] targeted aircraft control with FPGA-in-the-loop simulation; Shukla *et al.* [35] proposed a code-generation framework for FPGA splitting methods; and Jeong *et al.* [36] implemented ADMM for power electronics. To the best of our knowledge, all prior FPGA-based MPC implementations either target *explicit* MPC [37], which precomputes control laws entirely offline, use development boards too large or power-hungry for airborne deployment, or validate only in simulation or hardware-in-the-loop. As such, none have been deployed and flown onboard an aerial robot. Similarly, while the neural thrust controller of Azem *et al.* [38] showed that an FPGA can be flown on a Crazyflie, it does not perform online optimization.

### IV. HARDWARE-ALGORITHM CODESIGN

To the best of our knowledge, no prior work combines FPGA-accelerated *online* MPC with onboard deployment and closed-loop flight validation on a tiny aerial robot. Our end-to-end hardware-algorithm co-designed system fills this gap and enables constrained MPC at 1 kHz rates.

Achieving this requires satisfying the coupled control, optimization, compute, and physical-integration requirements summarized in Table I. High-bandwidth flight requires predictable kilohertz-rate feedback, while constrained MPC requires sufficient solver iterations and runtime-updatable constraints within each control cycle. FPGA acceleration is well suited to this setting because it provides programmable logic and arithmetic resources that can be organized into deterministic, deeply pipelined datapaths for fast and low-power computation without requiring chip fabrication. However, embedded FPGAs provide limited on-chip memory, and efficient implementations typically favor regular dataflow and fixed-point arithmetic over general-purpose floating-point computation. The resulting solver must therefore combine regular computation, compact storage, reduced-precision robustness, and efficient runtime adaptation.

AccelMPC therefore jointly designs the optimization algorithm, numerical representation, hardware mapping, and physical integration around these requirements. This resulted in two custom FPGA solver mappings (Sections IV-A and IV-B) and a custom PCB (Section IV-C) for deployment on a Crazyflie. We open source the entire AccelMPC project on our lab GitHub including: PCB design files, modified firmware, and the FPGA solver software (links redacted for double blind review).

#### A. Algorithm and Numerical Co-Design

At the algorithm level, the requirements in Table I favor a solver with regular dataflow, deterministic execution, robustness to reduced precision, and inexpensive runtime updates.

The ADMM algorithm presented in Sec. II-B provides these properties through its structured computational patterns and known robustness to numerical error [17], [39]. We

TABLE I: Requirements for fast, low-power, onboard constrained MPC on tiny aerial robots.

| Source            | Requirement                            |
|-------------------|----------------------------------------|
| Robotics          | Predictable 1 ms control period        |
| Constrained MPC   | Sufficient online iteration budget     |
| Online operation  | Runtime constraint updates             |
| Tiny aerial robot | Low power, mass, and footprint         |
| Fast, low-power   | FPGA or ASIC compute                   |
| Embedded FPGA     | Low memory and reduced precision       |
| FPGA execution    | Regular, pipeline-friendly computation |
| Integration       | Low-latency MCU-FPGA communication     |

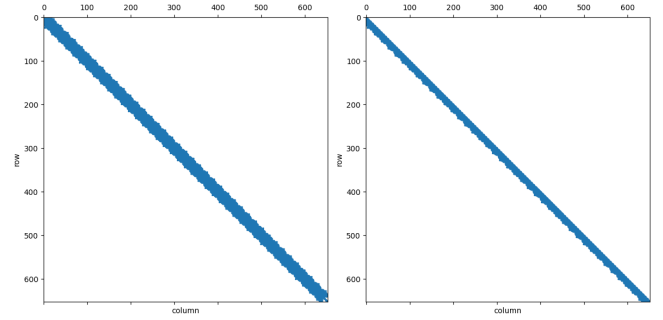


Fig. 2: Left: coefficient matrix of the  $w$ -update. Right: corresponding Cholesky factor. The banded sparsity structure is preserved, enabling streaming forward and backward substitution with complexity scaling linearly in the number of optimization variables.

therefore adopt the cached formulation of TinyMPC [16], trading a small amount of runtime flexibility for drastically improved computational performance. In particular, the matrices  $P$  and  $M$  and the penalty parameter  $\rho$  are fixed and prepared offline, only the initial state, the reference window, and the bound vectors  $l$  and  $u$  change at runtime. Finally, the auxiliary and dual variables are warm started across control cycles to exploit the temporal coherence between the closely spaced problems that arise at 1 kHz.

Two aspects of the solver must be redesigned for efficient execution on an embedded FPGA: the structured linear solve (7), which dominates the cost of each ADMM iteration, and the numerical representation.

TinyMPC evaluates the structured linear solve (7) through cached Riccati-derived gain and cost-to-go matrices, executing a stage-wise backward-forward recursion of small dense matrix-vector products. This introduces a loop-carried dependency at matrix granularity, a poor match for a pipelined FPGA datapath. Prior GPU-accelerated solutions instead precompute and store the full matrix inverse [40], however this exceeds the on-chip memory of an embedded FPGA.

We instead precompute a Cholesky factorization that exploits the banded structure of the coefficient matrix  $P + \rho M^\top M$  (Fig. 2). If the bandwidth is  $h$ , each row stores only  $h$  values, and the online solve reduces to a forward and a backward substitution pass whose loop-carried dependencies are at scalar granularity within a window of fixed depth

$h$ . Since  $h$  is determined by the local stage coupling and is nearly independent of the prediction horizon, the cost per ADMM iteration scales linearly with the number of optimization variables,  $\mathcal{O}(N_{\text{var}})$ , and the substitution passes can be streamed in a hardware pipeline (Sec. IV-B).

The second redesign is the numerical representation. Most critically, the solver operates in 32-bit fixed-point arithmetic, which lowers both power and latency relative to floating point on FPGA fabric. As our HLS implementation remains modular with respect to the numerical datatype, we validated our fixed-point implementation against an otherwise identical IEEE-754 single-precision implementation in simulation. Across the tested prediction horizons and iteration budgets, fixed-point quantization introduced negligible additional error, with differences of approximately  $10^{-6}$  m or less in predicted position RMSE,  $10^{-7}$  m in maximum constraint violation, and  $10^{-7}$  in control RMSE. We also restricted the penalty parameter,  $\rho$ , to a power of two, sacrificing some tuning freedom so that every multiplication by  $\rho$  or  $\rho^{-1}$  reduces to a left or right shift. Finally, the reciprocal of each diagonal element is stored in the final slot of its row of the Cholesky factor, eliminating all online division from the substitution passes. Together these choices reduce every arithmetic operation executed online to an addition, a multiplication, a shift, or a comparison, which is precisely the efficient operation set on embedded FPGAs.

Algorithm 1 summarizes the resulting online solve. We present it in the scaled-dual implementation form described above, where the right-hand side is maintained incrementally across iterations rather than reconstructed, and each line is annotated with the hardware structure that realizes it, which Sec. IV-B describes. Finally, the solver executes a fixed number of ADMM iterations each control step to ensure deterministic latency.

### B. FPGA Datapath and Complementary Mappings

Every line of Algorithm 1 maps to a streaming hardware structure. The substitution passes are implemented with a *sliding-window shift register* of depth  $h$  where newly computed values enter the window as old values shift out, so the computation operates on small local storage with fully regular access patterns that high-level-synthesis frameworks such as Vitis HLS [41] readily pipeline. The box projection reduces constraint handling to independent comparison-and-saturation operations, preserving a regular datapath even when constraints are active. The scaled-dual and right-hand-side updates are fused multiply-add streams over the same stage-wise structure, producing the next solve right-hand side without assembling dense matrices online.

One design axis remains open in this datapath, namely, how the constraint operators are represented. That is, they can either be stored explicitly in on-chip memory as expanded sparse operators, or the required coefficients can be regenerated from the underlying stage-wise system dynamics according to a deterministic access schedule. We implement both, as two FPGA mappings of the same solver, denoted `full_sparse` and `staged`, respectively. Both use the

---

### Algorithm 1 Co-designed fixed-iteration ADMM solve for online MPC

---

**Offline:** *Cache compiled into the bitstream:*  $M$ ; banded Cholesky factor,  $L$ , with reciprocal diagonals;  $\rho$ ; table of reference linear terms,  $c$ ; default nominal bounds  $l, u$

**Online:** *Dynamic Inputs:* state estimate  $\hat{x}$ ; trajectory index; optional bound override; iteration budget  $K$

---

- 1: Initial-condition  $\leftarrow \hat{x}$
  - 2: Select  $c$  for the current window ▷ table lookup
  - 3: Decode and apply any runtime bound override to  $(l, u)$
  - 4: Warm start  $(s, b)$  from the previous control cycle
  - 5: **for**  $j = 1, \dots, K$  **do**
  - 6:    $r \leftarrow b - c$  ▷ elementwise subtraction
  - 7:   Solve  $Lz = r$ , then  $L^\top w = z$  ▷ streamed, depth  $h$
  - 8:    $v \leftarrow Mw$  ▷ sparse or staged product
  - 9:    $y \leftarrow \Pi_{[l, u]}(v + s)$  ▷ compare-saturate
  - 10:    $s \leftarrow s + v - y$  ▷ elementwise add-subtract
  - 11:    $d \leftarrow \rho(y - s)$  ▷ shift
  - 12:    $b \leftarrow M^\top d$  ▷ sparse or staged product
  - 13: Store  $(s, b)$  for the next control cycle
  - 14: **return** first control input extracted from  $w$
- 

same fixed-point representation, execute the same ADMM updates, and expose the same runtime interface. They differ only in operator representation, placing the dominant hardware cost at different points in the memory-computation tradeoff. As these generate tradeoffs in energy and latency (Sec. V), no single choice is optimal across the full range of robotic MPC problem sizes or applications.

The `full_sparse` mapping's direct expanded storage avoids online reconstruction and achieves the lowest measured energy per solve. However, its scalability is ultimately limited by the horizon-dependent storage of the expanded matrices, which increases BRAM demand and eventually causes spillover into distributed LUT memory.

The `staged` mapping instead retains a compact stage-wise representation of the MPC dynamics and reconstructs the required operator products during execution. Despite this additional reconstruction, its more streamlined data movement yields slightly lower latency at short horizons in our implementation. More importantly, its memory requirements grow substantially more slowly with problem size, allowing much larger horizons to fit on the same FPGA. This improved scalability comes at the cost of moderately higher power and energy consumption over the shared operating range.

### C. Custom PCB Design and Integration

We found that existing solutions to mount an FPGA on the Crazyflie were unsuitable for kilohertz-rate onboard optimization. Available decks either provide insufficient FPGA resources (e.g., [42]) or rely on low-bandwidth interfaces such as I<sup>2</sup>C (e.g., [38]), which impose excessive communication latency. We therefore designed and fabricated a custom four-layer PCB expansion board (Fig. 3).

The board hosts an AMD Artix-7 100T FPGA powered by a dedicated PMIC that generates and sequences the re-



quired core and I/O voltage rails. A high-speed SPI interface connects the FPGA directly to the Crazyflie MCU, enabling low-latency exchange of the current optimization data and the resulting control input. A 100 MHz oscillator provides the solver clock, while an onboard SPI flash stores the FPGA configuration for autonomous startup. Additional JTAG access and user LEDs support programming and debugging.

To enable the integration of our custom PCB, we modified the FreeRTOS-based Crazyflie firmware to compute the deviation between the EKF state estimate and the current reference, transmit the runtime solver data to the FPGA, trigger the optimization, and retrieve the first optimal control input for actuation. Runtime constraint bounds can be updated through the same interface, allowing the admissible set to change from one control cycle to the next and supporting dynamic obstacle-avoidance (e.g., Sec. V-D). As a result, the complete ADMM solve is offloaded to the FPGA, while state estimation, communication, safety logic, and the remaining flight stack remain on the MCU. This division allows the accelerator to be integrated without replacing the established Crazyflie autonomy stack.

## V. EXPERIMENTAL EVALUATION

### A. Methodology

We evaluate the proposed solver in three complementary settings. First, we benchmark TinyMPC on the Crazyflie MCU (STM32F405, 168 MHz) with the FPGA implementations over prediction horizons  $H \in \{10, 20, 30, 40, 50, 60, 70, 80\}$ . TinyMPC is measured inside the FreeRTOS flight stack over 80 samples per operating point, whereas FPGA execution is cycle-deterministic and therefore requires only a single measurement per synthesized configuration. Second, we compare in more detail the two FPGA mappings, *full\_sparse* and *staged*, over the range where both implementations fit on the target Artix-7 FPGA and discuss the scalability of the *staged* mapping, extending the prediction horizon to  $H = 1350$ . Finally, we deploy our approach onto a Crazyflie Brushless 2.1 in an OptiTrack eight-camera arena with a 120 Hz position stream. The ADMM control loop is fully offloaded to the onboard FPGA and runs at 1 kHz. During flight, the remaining standard Crazyflie flight stack executes on the onboard MCU.

### B. Solver Benchmarks

Fig. 4 shows the solve time versus prediction horizon, along with the energy per solve, for TinyMPC and both the *full\_sparse* and *staged* FPGA implementations. While all three implementations scale approximately linearly with horizon, the FPGA implementations are much faster and consume far less energy. In particular, the FPGA implementations are up to  $15.6\times$  faster than TinyMPC. The median time per ADMM iteration per horizon step is  $52.3\mu\text{s}/(\text{iter}\cdot\text{step})$  for TinyMPC,  $3.69\mu\text{s}/(\text{iter}\cdot\text{step})$  for *full\_sparse*, and  $3.33\mu\text{s}/(\text{iter}\cdot\text{step})$  for *staged*. Similarly, the FPGA implementations achieve  $5.8\text{--}14.0\times$  lower energy per solve and  $79.1\text{--}195.4\times$  lower energy-delay product (EDP) than TinyMPC (Fig. 5). As expected, *full\_sparse* uses far

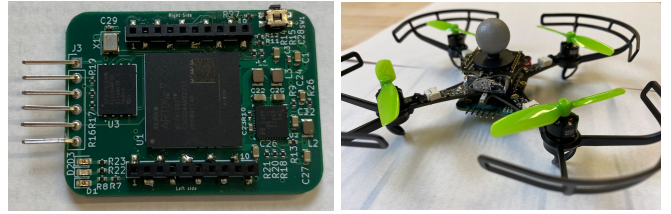


Fig. 3: The  $4\times 3$  cm four-layer PCB hosts an AMD Artix-7 100T FPGA and mounts directly on the Crazyflie through the standard expansion interface. Left: fabricated FPGA board. Right: fully integrated onboard the tiny quadrotor.

less power but is unable to scale as well as *staged*. For TinyMPC, MCU power consumption is evaluated under active operational parameters (87 mA at 168 MHz, 3.3 V), establishing a nominal compute power profile for the STM32F405. FPGA power is obtained from post-route power analysis combined with the measured solve time.

Table II also reports the maximum feasible ADMM iteration budget within the target 1 kHz control period, including MCU-FPGA communication overhead. We find that within a 1 kHz control period, TinyMPC cannot sustain a sufficient optimization budget for constrained MPC even at the shortest prediction horizons, and quickly becomes unable to complete a single ADMM iteration as the horizon increases. The FPGA implementations instead preserve multiple iterations across the evaluated horizons, making kilohertz-rate constrained MPC practical on the target platform.

### C. FPGA Architecture Tradeoffs and Scalability

We next compare the *full\_sparse* and *staged* FPGA mappings in additional detail. Overall, as shown in Figure 6, neither mapping dominates across all design objectives and both provide high performance and low power control.

As noted above, Fig. 4 shows that both implementations have nearly identical latency scaling over their shared operating range, with the *staged* mapping consistently providing slightly lower latency. In contrast, *full\_sparse* achieves lower power and energy per solve in the horizon range where its expanded operators fit efficiently in on-chip memory.

These performance differences reflect the underlying resource usage. The *full\_sparse* mapping prioritizes specialized datapaths and lower compute utilization at the expense of increasing BRAM and LUTRAM pressure as the horizon grows. Conversely, *staged* trades additional computation for a substantially more compact memory footprint, allowing the implementation to avoid the LUTRAM spillover that limits *full\_sparse* near  $H = 90$ . This compact footprint allows *staged* to scale to  $H = 1350$ , corresponding to 21,612 optimization variables and 24,312 constraints, while maintaining a solve time of 44.9 ms for  $k = 10$  ADMM iterations, equivalent to 22.3 online solves per second. Although this operating point targets slower robotic control or planning loops rather than kilohertz-rate quadrotor control, it demonstrates that the same solver architecture extends well beyond the embedded flight regime without algorithmic modifications.

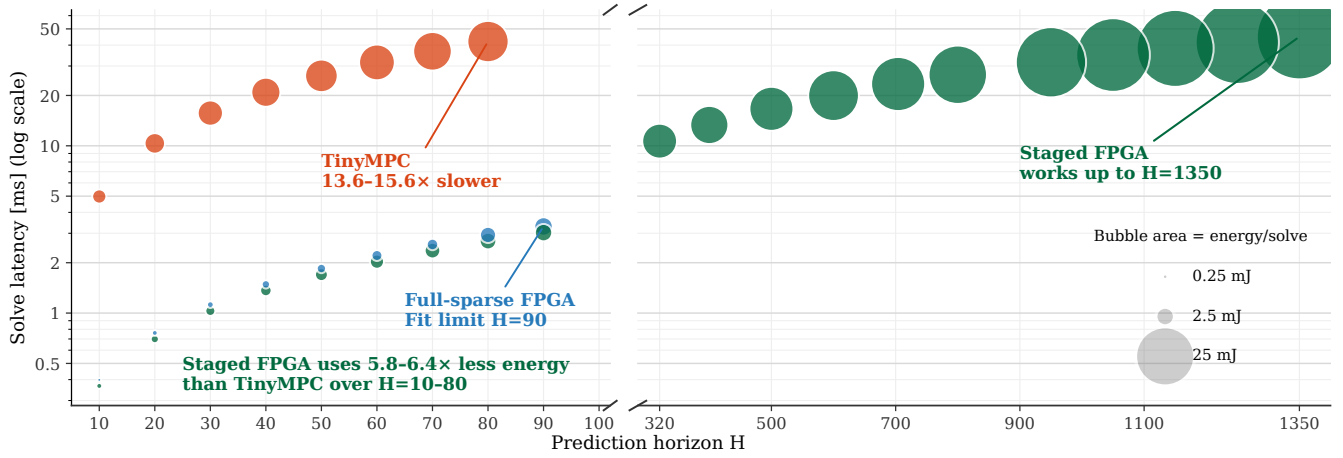


Fig. 4: Latency and energy versus prediction horizon for the staged FPGA design, full\_sparse FPGA baseline, and TinyMPC at a fixed ADMM budget of  $k=10$ . Marker area is proportional to energy per solve.

TABLE II: Maximum feasible ADMM iterations within a 1 kHz control period, FPGA includes 100  $\mu$ s SPI overhead.

| Solver           | Prediction Horizon $H$ |    |    |    |    |    |    |    |     |     |     |     |     |
|------------------|------------------------|----|----|----|----|----|----|----|-----|-----|-----|-----|-----|
|                  | 5                      | 10 | 20 | 30 | 40 | 50 | 70 | 90 | 100 | 150 | 200 | 250 | 275 |
| TinyMPC (MCU)    | 4                      | 2  | —  | —  | —  | —  | —  | —  | —   | —   | —   | —   | —   |
| Full_sparse FPGA | 41                     | 22 | 11 | 8  | 6  | 4  | 3  | 2  | —   | —   | —   | —   | —   |
| Staged FPGA      | 43                     | 24 | 12 | 8  | 6  | 5  | 3  | 2  | 2   | 1   | 1   | 1   | —   |

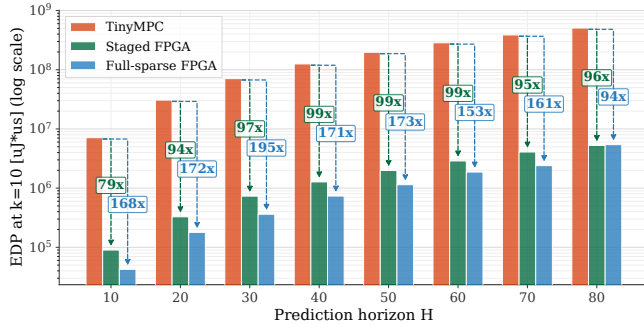


Fig. 5: Energy-delay product at 10 ADMM iterations across various horizons. FPGA implementations achieve up to 195.4× lower EDP than TinyMPC.

#### D. Onboard Flight Experiments

1) *Unconstrained Figure-Eight Tracking*: We evaluate nominal tracking performance on a smooth figure-eight trajectory in the horizontal plane at constant altitude. TinyMPC uses  $H=15$  and  $k=2$  at 500 Hz, while the FPGA controller is able to scale to  $H=20$  and  $k=28$  due to its faster performance within the same control rate. As no constraints are active in this experiment, both configurations solve tractable QPs and achieve comparable closed-loop tracking performance, as shown in Fig. 7. This experiment verifies that the FPGA implementation preserves the nominal tracking behavior of the embedded baseline while providing a substantially larger online optimization budget.

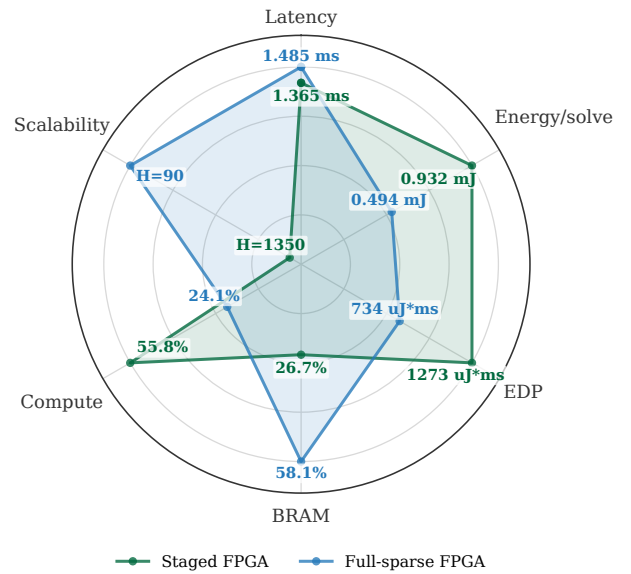


Fig. 6: A multi-objective comparison of the staged and full\_sparse FPGA solvers at  $H=40$  and  $k=10$  ADMM iterations. All metrics are scaled so that lower is better.

2) *Static constrained trajectory tracking*: We restrict the admissible workspace to a square region via box constraints on the  $x$  and  $y$  coordinates, and design a star-shaped reference that intentionally violates these bounds. This forces the controller to modify the commanded motion online to maintain feasibility.

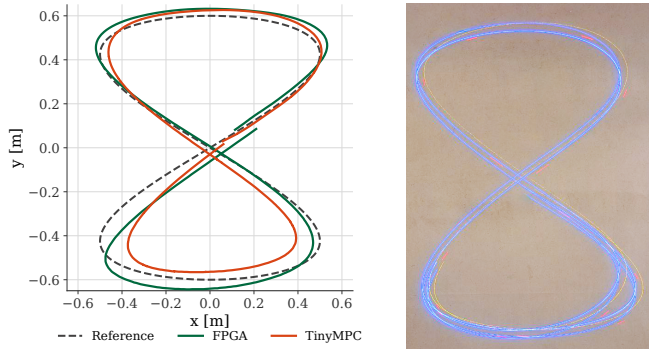


Fig. 7: Unconstrained figure-eight trajectory tracking. TinyMPC and the FPGA-based controller achieve comparable closed-loop tracking performance. Left: trajectory overlay in the  $xy$  plane. Right: long-exposure flight photograph.

Fig. 8 shows that the measured trajectory remains inside the admissible region throughout the physical flight. At 1 kHz, TinyMPC cannot execute enough ADMM iterations to reliably converge to a feasible constrained solution, even at short horizons. The FPGA implementation instead solves the constrained problem with  $H=27$  and  $k=9$  within every control cycle, enabling kilohertz-rate constraint enforcement onboard a tiny quadrotor.

3) *Dynamic obstacle avoidance*: Finally, we evaluate the ability of the controller to react online to dynamically moving obstacles without modifying the FPGA configuration. The quadrotor tracks a nominal figure-eight reference while two operators move instrumented boxes through the flight volume and across the nominal trajectory.

Each box is tracked as an OptiTrack rigid body, and its measured position is transmitted to the quadrotor at the motion-capture update rate of 120 Hz. Each obstacle is represented by a separating half-space with a fixed normal and an offset determined from the measured obstacle position and a prescribed safety margin. As such, obstacle motion changes only the corresponding entries of  $l$  and  $u$ , while the constraint matrix remains fixed. Thus, the precomputed Cholesky factorization and the FPGA bitstream remain unchanged throughout the experiment, allowing the onboard controller to incorporate these updated bounds while continuing to solve the MPC problem at 1 kHz.

As shown in Fig. 1, the quadrotor follows the nominal figure-eight trajectory while the path is unobstructed. When either box moves into the nominal flight path, the obstacle-dependent constraints modify the admissible region, causing the optimized trajectory to deviate from the reference and avoid the obstacle. Once the path becomes clear, the quadrotor returns to the nominal trajectory.

## VI. CONCLUSION AND FUTURE WORK

We presented AccelMPC, an open source, FPGA-accelerated linear MPC controller paired with a custom PCB for fast, low-latency communication. AccelMPC achieves up to a  $15.6\times$  speedup and  $195.4\times$  improvement in energy-delay product over state-of-the-art embedded MCU solvers,

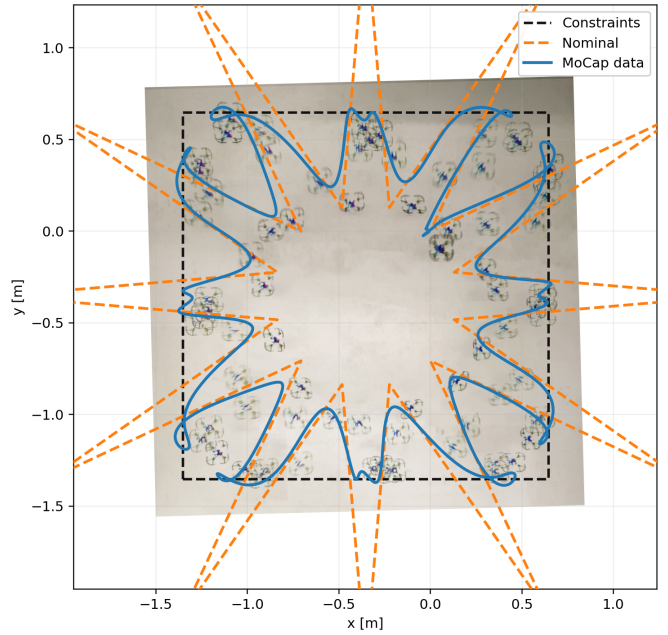


Fig. 8: Hardware demonstration of constrained trajectory tracking at 1 kHz. Box constraints on  $x$  and  $y$  are enforced fully onboard by the FPGA-accelerated MPC controller ( $H = 27, k = 9$ ). The trajectory remains within the admissible region despite a constraint-violating reference.

enables 1 kHz constrained trajectory tracking on a tiny quadrotor, and scales to problems with over 20,000 optimization variables on the same solver architecture.

Future work will extend the system to richer collision-avoidance formulations and more aggressive flight regimes. Cluttered and fully three-dimensional environments, online replanning around moving obstacles, and drone-racing-style trajectories introduce more complex geometric and dynamic constraints while pushing the vehicle closer to its actuation limits [4], [5], [10], [14], [15], [43]. These settings provide a natural testbed for evaluating how high-rate hardware-accelerated control can support more sophisticated models and constraints under the stringent compute and power budgets of tiny aerial robots. Future work will also more systematically analyze the numerical effects of reduced-precision arithmetic, for example using automated precision-analysis tools such as RoboPrec [44], with the goal of establishing accuracy and constraint-satisfaction guarantees for the current fixed-point implementation. Building on these guarantees, more aggressive fixed-point representations could then be explored to further reduce FPGA resource usage and power while preserving closed-loop control performance.

## REFERENCES

- [1] K. N. McGuire, C. De Wagter, K. Tuyls, H. J. Kappen, and G. C. de Croon, “Minimal navigation solution for a swarm of tiny flying robots to explore an unknown environment,” *Science Robotics*, vol. 4, no. 35, p. eaaw9710, 2019.
- [2] B. P. Duisterhof, S. Li, J. Burgués, V. J. Reddi, and G. C. De Croon, “Sniffy bug: A fully autonomous swarm of gas-seeking nano quadcopters in cluttered environments,” in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021.

- [3] X. Zhou, X. Wen, Z. Wang, Y. Gao, H. Li, Q. Wang, T. Yang, H. Lu, Y. Cao, C. Xu, *et al.*, "Swarm of micro flying robots in the wild," *Science robotics*, vol. 7, no. 66, p. eabm5954, 2022.
- [4] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, no. 7976, pp. 982–987, 2023.
- [5] D. Hanover, A. Loquercio, L. Bauersfeld, A. Romero, R. Penicka, Y. Song, G. Cioffi, E. Kaufmann, and D. Scaramuzza, "Autonomous drone racing: A survey," *IEEE Trans. Robot.*, vol. 40, 2024.
- [6] S. M. Neuman, B. Plancher, B. P. Duisterhof, S. Krishnan, C. Banbury, M. Mazumder, S. Prakash, J. Jabbour, A. Faust, G. C. de Croon, *et al.*, "Tiny robot learning: Challenges and directions for machine learning in resource-constrained robots," in *IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, 2022.
- [7] J. Rawlings, D. Mayne, and M. Diehl, *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2017.
- [8] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, "Constrained model predictive control: Stability and optimality," *Automatica*, vol. 36, no. 6, pp. 789–814, June 2000.
- [9] E. F. Camacho and C. Bordons, *Model Predictive Control*, ser. Advanced Textbooks in Control and Signal Processing, M. J. Grimble and M. A. Johnson, Eds. London: Springer, 2007.
- [10] M. Kamel, J. Alonso-Mora, R. Siegwart, and J. Nieto, "Robust collision avoidance for multiple micro aerial vehicles using nonlinear model predictive control," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Sept. 2017.
- [11] D. Falanga, P. Foehn, P. Lu, and D. Scaramuzza, "Pampc: Perception-aware model predictive control for quadrotors," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 1–8.
- [12] G. Torrente, E. Kaufmann, P. Föhn, and D. Scaramuzza, "Data-driven MPC for quadrotors," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3769–3776, 2021.
- [13] A. Romero, S. Sun, P. Foehn, and D. Scaramuzza, "Model predictive contouring control for time-optimal quadrotor flight," *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3340–3356, 2022.
- [14] A. Romero, R. Penicka, and D. Scaramuzza, "Time-optimal online replanning for agile quadrotor flight," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 7730–7737, July 2022.
- [15] M. Krinner, A. Romero, L. Bauersfeld, M. Zeilinger, A. Carron, and D. Scaramuzza, "MPCC++: Model predictive contouring control for time-optimal flight with safety constraints," in *Robotics: Science and Systems XX*, July 2024.
- [16] K. Nguyen, S. Schoedel, A. Alavilli, B. Plancher, and Z. Manchester, "Tinympc: Model-predictive control on resource-constrained micro-controllers," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 1–7.
- [17] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers," *Foundations and Trends in Machine Learning*, vol. 3, pp. 1–122, Jan. 2011.
- [18] I. Mahajan, K. Nguyen, S. Schoedel, E. Nedumaran, M. Mata, B. Plancher, and Z. Manchester, "Code generation and conic constraints for model-predictive control on microcontrollers with conic-tinympc," in *IEEE International Conference on Robotics and Automation (ICRA)*, June 2026.
- [19] I. Mahajan and B. Plancher, "Robust and efficient embedded convex optimization through first-order adaptive caching," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025.
- [20] I. Mahajan, J. Arrizabalaga, A. Grillo, F. Vega, J. Anderson, Z. Manchester, and B. Plancher, "Tinysdp: Real time semidefinite optimization for certifiable and agile edge robotics," in *Robotics Science and Systems (RSS)*, Sydney, Australia, July 2026.
- [21] H. J. Ferreau, S. Almér, R. Verschueren, M. Diehl, D. Frick, A. Domahidi, J. L. Jerez, G. Stathopoulos, and C. Jones, "Embedded Optimization Methods for Industrial Automatic Control," *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 13 194–13 209, July 2017.
- [22] J. Mattingley and S. Boyd, "CVXGEN: A code generator for embedded convex optimization," *Optimization and Engineering*, vol. 13, Mar. 2012.
- [23] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "OSQP: An operator splitting solver for quadratic programs," *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, Dec. 2020.
- [24] G. Frison and M. Diehl, "Hpipm: a high-performance quadratic programming framework for model predictive control," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 6563–6569, 2020.
- [25] R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, and M. Diehl, "acados – a modular open-source framework for fast embedded optimal control," *Mathematical Programming Computation*, 2021.
- [26] B. E. Jackson, T. Punnoose, D. Neamati, K. Tracy, R. Jitosh, and Z. Manchester, "Altro-c: A fast solver for conic model-predictive control," in *IEEE International Conference on Robotics and Automation (ICRA)*, Xi'an, China, May 2021.
- [27] M. Schubiger, G. Banjac, and J. Lygeros, "GPU acceleration of ADMM for large-scale quadratic programming," *Journal of Parallel and Distributed Computing*, vol. 144, pp. 55–67, Oct. 2020.
- [28] E. Adabag, M. Atal, W. Gerard, and B. Plancher, "MPCGPU: Real-Time Nonlinear Model Predictive Control through Preconditioned Conjugate Gradient on the GPU," in *IEEE International Conference on Robotics and Automation*, 2024, pp. 9787–9794.
- [29] L. Amatucci, J. Sousa-Pinto, G. Turrissi, D. Orban, V. Barasuol, and C. Semini, "Primal-dual ilqr for gpu-accelerated learning and control in legged robots," *IEEE Robotics and Automation Letters*, vol. 11, no. 1, pp. 1010–1017, 2025.
- [30] W. Giernacki, M. Skwierczyński, W. Witwicki, P. Wroński, and P. Kozierski, "Crazyflie 2.0 quadrotor as a platform for research and education in robotics and control engineering," in *2017 22nd International Conference on Methods and Models in Automation and Robotics (MMAR)*. IEEE, 2017, pp. 37–42.
- [31] I. McInerney, G. A. Constantinides, and E. C. Kerrigan, "A Survey of the Implementation of Linear Model Predictive Control on FPGAs," *IFAC-PapersOnLine*, vol. 51, no. 20, pp. 381–387, Jan. 2018.
- [32] J. L. Jerez, P. J. Goulart, S. Richter, G. A. Constantinides, E. C. Kerrigan, and M. Morari, "Embedded predictive control on an FPGA using the fast gradient method," in *IEEE European Control Conference (ECC)*, 2013.
- [33] —, "Embedded Online Optimization for Model Predictive Control at Megahertz Rates," *IEEE Transactions on Automatic Control*, vol. 59, no. 12, pp. 3238–3251, Dec. 2014.
- [34] E. N. Hartley, J. L. Jerez, A. Suardi, J. M. Maciejowski, E. C. Kerrigan, and G. A. Constantinides, "Predictive Control Using an FPGA With Application to Aircraft Control," *IEEE Transactions on Control Systems Technology*, vol. 22, no. 3, pp. 1006–1017, May 2014.
- [35] H. A. Shukla, B. Khusainov, E. C. Kerrigan, and C. N. Jones, "Software and Hardware Code Generation for Predictive Control Using Splitting Methods\*," *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 14 386–14 391, July 2017.
- [36] M. Jeong, M. Schoen, and J. Biela, "When FPGAs Meet ADMM with High-level Synthesis (HLS): A Real-time Implementation of Long-horizon MPC for Power Electronic Systems," in *2023 11th International Conference on Power Electronics and ECCE Asia (ICPE 2023 - ECCE Asia)*, May 2023, pp. 1704–1711.
- [37] P. Tondel, T. Johansen, and A. Bemporad, "Computation and approximation of piecewise affine control laws via binary search trees," in *Proceedings of the 41st IEEE Conference on Decision and Control*, 2002., vol. 3, Dec. 2002, pp. 3144–3149 vol.3.
- [38] S. Azem, D. Scheunert, M. Li, J. Gehringer, K. Cui, C. Hochberger, and H. Koeppl, "FPGA-Based Neural Thrust Controller for UAVs," <https://arxiv.org/abs/2403.18703v2>, Mar. 2024.
- [39] J. Eckstein and D. P. Bertsekas, "On the douglas—rachford splitting method and the proximal point algorithm for maximal monotone operators," *Mathematical programming*, vol. 55, pp. 293–318, 1992.
- [40] A. L. Bishop, J. Z. Zhang, S. Gurumurthy, K. Tracy, and Z. Manchester, "Relu-qp: A gpu-accelerated quadratic programming solver for model-predictive control," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 13 285–13 292.
- [41] AMD, Inc., *Vitis High-Level Synthesis User Guide (UG1399)*, Advanced Micro Devices (AMD), 2026, version 2025.2.
- [42] Bitcraze AB, "Lighthouse positioning deck," <https://store.bitcraze.io/products/lighthouse-positioning-deck>, accessed: 2026-04-08. Features an onboard Lattice iCE40UP5K FPGA.
- [43] R. Penicka and D. Scaramuzza, "Minimum-time quadrotor waypoint flight in cluttered environments," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 5719–5726, Apr. 2022.
- [44] A. E. Yilmaz, T. Bourgeat, L. Pentecost, B. Plancher, and S. M. Neuman, "Roboprec: Enabling reliable embedded computing for robotics by providing accuracy guarantees across mixed-precision datatypes," *IEEE Robotics and Automation Letters*, vol. 11, pp. 2234–2241, 2026.